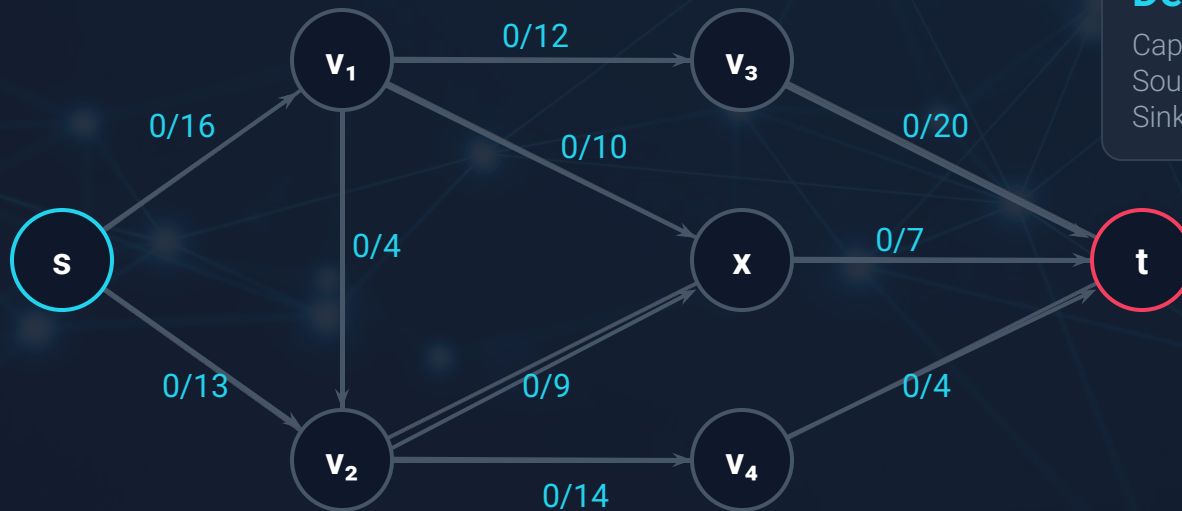


Maximum Flow and Edmonds-Karp

Luke Kaiser

Algorithm Analysis & Design

Introduction & Recap



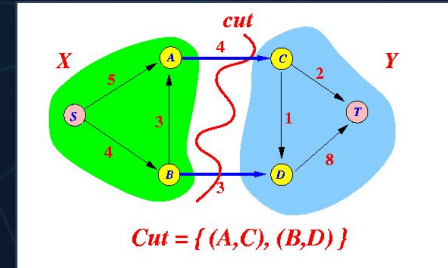
Definitions

Capacity: $c(u,v) \geq 0$
Source (s): Producer
Sink (t): Consumer

A directed graph $G = (V, E)$ where each edge has a stated maximum flow rate.

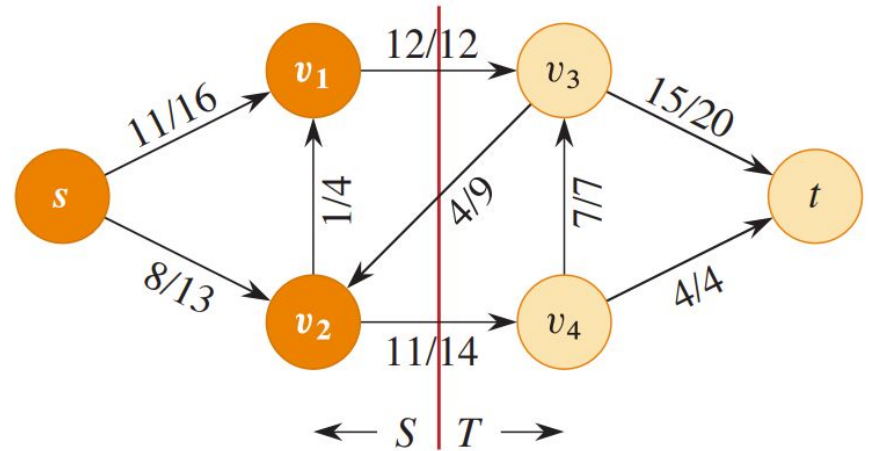
Cuts of Flow Networks

- A cut (S, T) of a flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that source $s \in S$ and sink $t \in T$.
- The net flow across the cut is $f(S, T) = \sum_{u \in S} \sum_{v \in T} f(u, v) - \sum_{u \in S} \sum_{v \in T} f(v, u)$
- The capacity of the cut is $c(S, T) = \sum_{u \in S} \sum_{v \in T} c(u, v)$
- The value of any flow f in a flow network G is bounded from above by the capacity of any cut of G ($|f| \leq c(s, t)$)

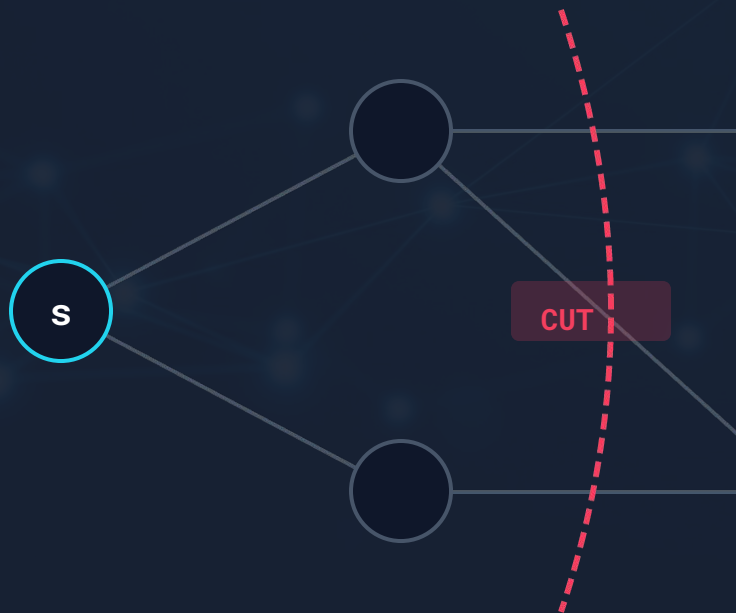


Min Cuts of Flow Networks

- A minimum cut of a network is a cut whose capacity is minimum over all cuts of the network.
- In this example, The net flow across this cut is $f(v_1, v_3) + f(v_2, v_4) - f(v_3, v_2) = 12 + 11 - 4 = 19$
- And the capacity of this cut is $c(v_1, v_3) + c(v_2, v_4) = 12 + 14 = 26$



Max-Flow Min-Cut Theorem



The theorem states that the **Max-Flow** through a network is equal to the **Min-Cut** capacity.

A cut partitions vertices into two sets S (source) and T (sink).

$$|f| = c(S, T)$$

Equality holds when all forward edges across the cut are saturated and backward edges are zero.

Termination criteria: The algorithm ends when no augmenting paths exist in G_f , indicating we've found the minimum cut.

The Max-Flow Min-Cut Theorem (Theorem 24.6)

If f is a flow in a flow network $G=(V,E)$ with source s and sink t , then the following conditions are equivalent:

1. f is a maximum flow in G
2. The residual network G_f contains no augmenting paths
3. $|f| = c(S,T)$ for some cut (S,T) of G

Proof of Max-Flow Min-Cut (1 implies 2)

Proof (1) $\Rightarrow$ (2):

- Suppose for the sake of contradiction that f is a maximum flow in G but that G_f has an augmenting path p .
- Then, by Corollary 24.3, the flow found by augmenting f by f_p is a flow in G with value strictly greater than $|f|$
- This contradicts the assumption that f is a maximum flow.

Proof of Max-Flow Min-Cut (2 implies 3)

Proof (2) $\Rightarrow$ (3):

- Suppose that G_f has no augmenting path, that is, G_f contains no path from s to t .
- Define $S = \{v \in V : \text{there exists a path from } s \text{ to } v \text{ in } G_f\}$ and let $T = V - S$. The partition (S, T) is a cut.
- For any vertices $u \in S$ and $v \in T$, we must have $f(u, v) = c(u, v)$, otherwise $(u, v) \in E_f$ and v would be placed in set S .
- Likewise, $f(v, u) = 0$. We thus have $f(S, T) = \sum c(u, v) - \sum 0 = c(S, T)$.
- By Lemma 24.4, $|f| = c(S, T)$.

Slide: Proof of Max-Flow Min-Cut (3 implies 1)

Proof (3) $\Rightarrow$ (1):

- By Corollary 24.5, $|f| \leq c(S, T)$ for all cuts (S, T) .
- The condition $|f| = c(S, T)$ thus implies that f is a maximum flow.

The Edmonds-Karp Algorithm

The Edmonds-Karp algorithm is an implementation of the Ford-Fulkerson method where the augmenting path p is always chosen to be the shortest path from s to t in the residual network G_f

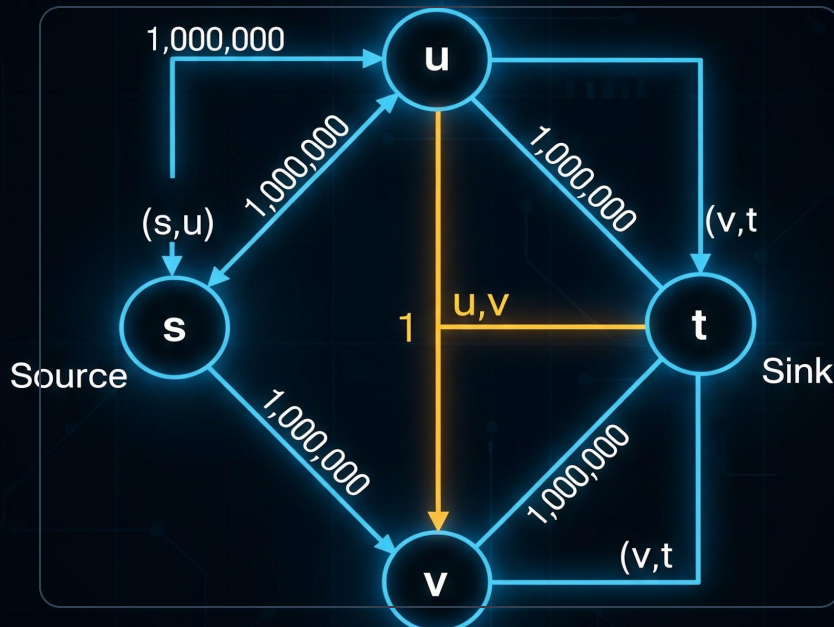
How it works: It finds this shortest path by running a Breadth-First Search (BFS), where each edge has unit distance.

Why it is better: Standard Ford-Fulkerson can take $O(E|f^*|)$ time, running very slowly on graphs with pathological capacities. Edmonds-Karp guarantees termination independent of the maximum flow value.

Space Complexity: $O(V+E)$ to store the graph and residual network.

Time Complexity: $O(VE^2)$ because BFS takes $O(E)$ and there are at most $O(VE)$ augmentations.

The Bad Case & Edmonds-Karp



Pathological Scenario

Alternating paths through the middle edge increase flow by only 1 unit/iteration.

Iterations: 2,000,000

Edmonds-Karp Fix

Mandates **Breadth-First Search (BFS)** to find the shortest path. Bounds complexity to:

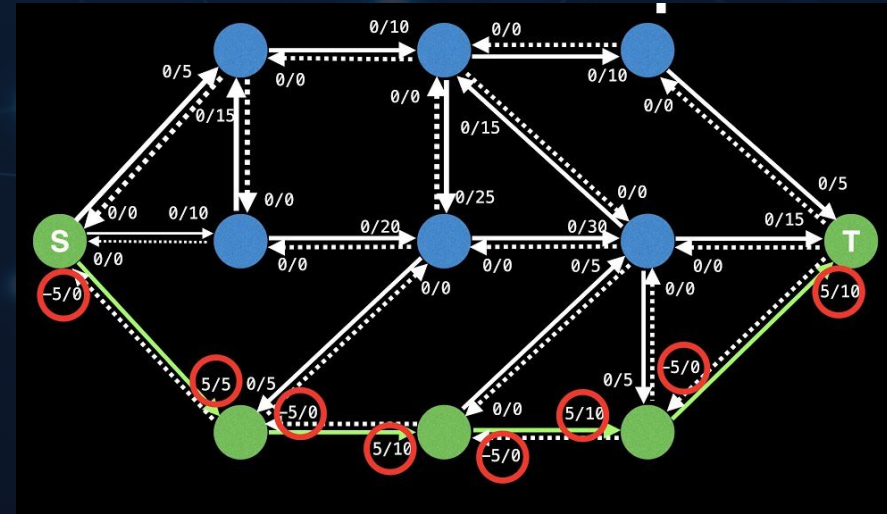
$$O(VE^2)$$

The Edmonds-Karp Algorithm

The Edmonds-Karp algorithm is an implementation of the Ford-Fulkerson method where the augmenting path p is always chosen to be the shortest path from s to t in the residual network G_f

How it works:

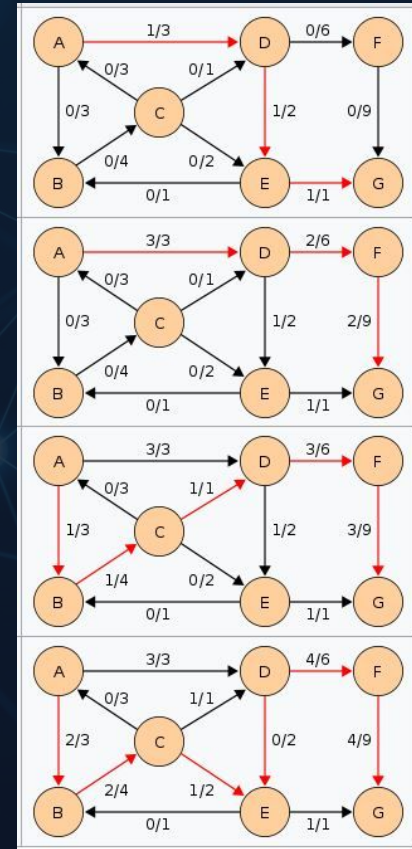
It finds this shortest path by running a Breadth-First Search (BFS), where each edge has unit distance.



The Edmonds-Karp Algorithm

Step by step:

- Initialize Flow: Set the flow of every edge in the original network to zero.
- Construct/Update Residual Graph: Build a residual graph where each edge has a residual capacity:
 - Forward edges: Remaining capacity ($c-f$)
 - backward edges: Amount of flow that can be "pushed back" (f).
- Find Shortest Path (BFS): Run a BFS on the residual graph to find the shortest path from the source s to the sink t
- If no path exists, the current total flow is the Maximum Flow, and the algorithm ends.
- Identify Bottleneck: In the found path, identify the edge with the smallest residual capacity. This value is the bottleneck capacity
- Augment Flow: Update the flow along the path
 - Add f to the flow of forward edges.
 - Subtract f from the flow of backward edges.
- Repeat: Go back to Step 2 until BFS can no longer find a path.



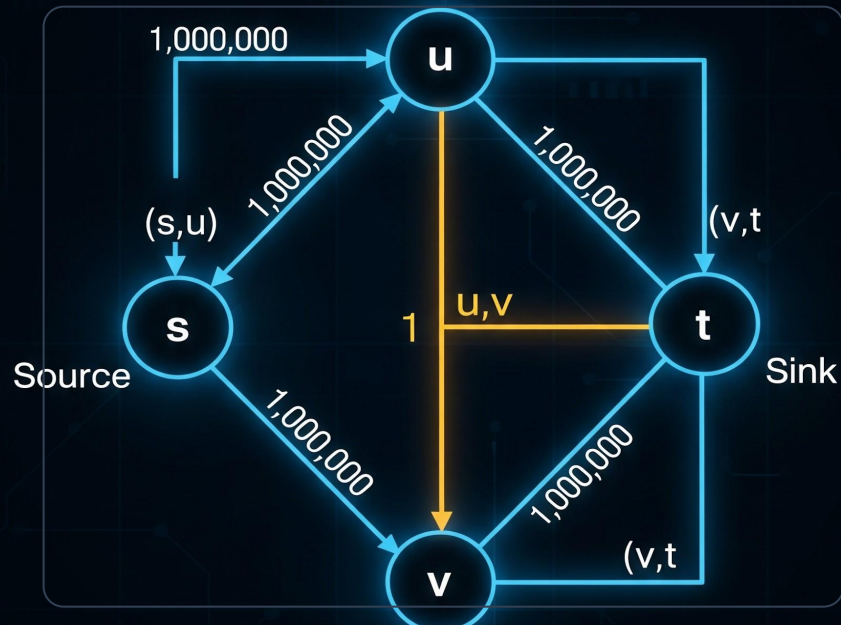
The Edmonds-Karp Algorithm (Cont.)

Why it is better: Standard Ford-Fulkerson can take $O(E|f^*|)$ time, running very slowly on graphs with pathological capacities. Edmonds-Karp guarantees termination independent of the maximum flow value.

Edmonds-Karp can help eliminate the worst case possibility of depth first search being reliant on edge weight.

Space Complexity: $O(V+E)$ to store the graph and residual network.

Time Complexity: $O(VE^2)$ because BFS takes $O(E)$ and there are at most $O(VE)$ augmentations.



Proof of Lemma 24.7 (Monotonicity of Shortest Paths)

If the Edmonds-Karp algorithm is run on a flow network $G = (V, E)$ with source s and sink t , then for all vertices $v \in V - \{s, t\}$, the shortest path distance $\delta_f(s, v)$ in the residual network G_f increases monotonically with each flow augmentation.

Prove this by contradiction.

- Let f be the flow just before an augmentation, and f' be the flow just after.
- Assume there is some vertex v whose shortest-path distance decreases (i.e., $\delta_{f'}(s, v) < \delta_f(s, v)$).
- Let v be the vertex with the minimum $\delta_{f'}(s, v)$ among all vertices whose distance decreased.

Proof of Lemma 24.7 (Contradiction)

- Let $p = s \rightarrow u \rightarrow v$ be the shortest path from s to v in G_f' .
- So for $\delta f'(s, v) = \delta f'(s, u) + 1$
- Because of how we chose v , we know u 's distance did not decrease: $\delta f'(s, u) \geq \delta f(s, u)$.
- If edge (u, v) was in the residual network G_f , then $\delta f(s, v) \leq \delta f(s, u) + 1 \leq \delta f'(s, u) + 1 = \delta f'(s, v)$. This contradicts our assumption that v 's distance decreased!
- Therefore, (u, v) was not in G_f . It was added during the augmentation.
- This only happens if the augmenting path pushed flow from v to u . Since Edmonds-Karp uses shortest paths, $\delta f(s, u) = \delta f'(s, v) + 1$
- Substituting this, we get $\delta f'(s, v) = \delta f'(s, u) + 1 \geq \delta f(s, u) + 1 = \delta f(s, v) + 2$.
- This contradicts $\delta f'(s, v) < \delta f(s, v)$. The lemma holds

Proof of Theorem 24.8 ($O(VE)$ Augmentations)

If the Edmonds-Karp algorithm is run on a flow network $G = (V, E)$, the total number of flow augmentations is $O(VE)$.

Proof Concept:

- An edge (u, v) in a residual network is a critical edge if its residual capacity equals the bottleneck capacity of the augmenting path.
- When an edge is critical, it disappears from the residual network in the next step.
- Every augmenting path must have at least one critical edge. We will count how many times any specific edge can become critical.

Proof of Theorem 24.8 ($O(VE)$ Bound)

Proof:

- When (u, v) is critical, it is part of the shortest path: $\delta f(s, v) = \delta f(s, u) + 1$
- It disappears from the residual network. It cannot reappear until flow is pushed back from v to u .
- If flow is pushed from v to u in a later flow f' , then (v, u) must be on the shortest path: $\delta f'(s, u) = \delta f'(s, v) + 1$
- By Lemma 24.7 (distances only increase), $\delta f'(s, v) \geq \delta f(s, v)$
- $\delta f'(s, u) = \delta f'(s, v) + 1 \geq \delta f(s, v) + 1 = \delta f(s, u) + 2$.
- From the time (u, v) becomes critical to the time it becomes critical again, the distance of u from the source increases by at least 2.
- Since distances are at most $|V| - 2$, an edge can be critical at most $|V|/2$ times.
- There are $2|E|$ total possible edges in G_f , so total critical edges (and total augmentations) is bounded by $O(VE)$.

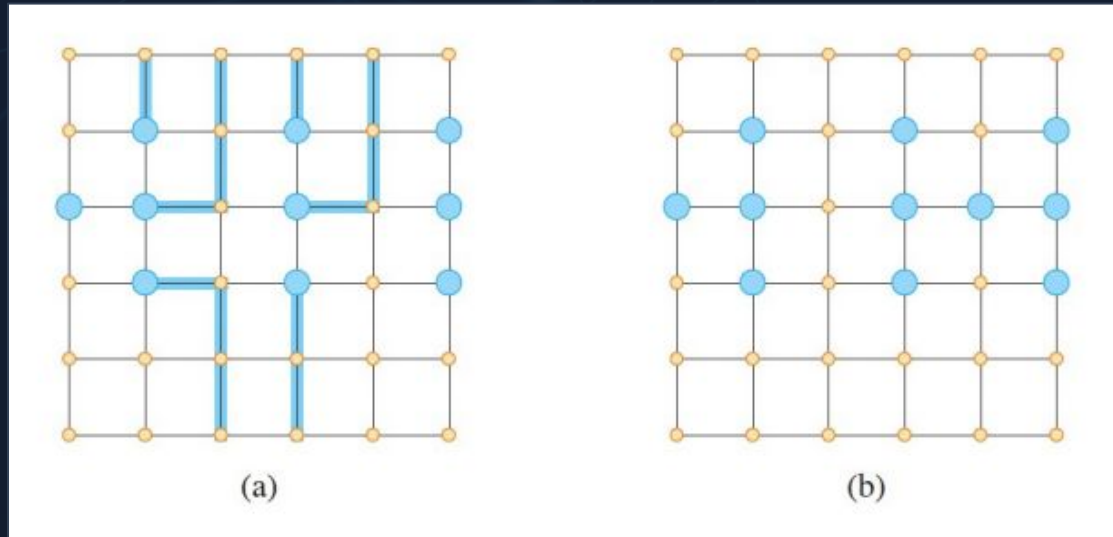
Problem 24-1: The Escape Problem (Part A)

We have an $n \times n$ grid with m starting points. Vertices have a capacity of 1 (a vertex can only be used once). We want to find if there are m vertex-disjoint paths to the grid boundaries.

Reduce determining the maximum flow in a network with vertex capacities to an ordinary flow network (with only edge capacities).

Figure 24.9 Grids for the escape problem. Starting points are blue, and other grid vertices are tan.

- (a) A grid with an escape, shown by blue paths.
(b) A grid with no escape.



Problem 24-1: The Escape Problem (Part A Solution)

Use Vertex Splitting.

- For every vertex v in the original graph G , we replace it with two new vertices in C , let's call them V_{in} and v_{out} .
- We add a directed edge from V_{in} to v_{out} with a capacity equal to the original vertex capacity (for the Escape Problem, $c(V_{in}, V_{out}) = 1$).
- For every original incoming edge (u, v) , we create an edge (U_{out}, V_{in}) in G' with capacity ∞ (or 1 for this grid).
- For every original outgoing edge (v, w) , we create an edge (V_{out}, W_{in}) in G' with capacity ∞ (or 1).
- Result: The vertex capacity constraint is perfectly modeled by the edge capacity between the split nodes.

Problem 24-1: The Escape Problem (Part B)

Build flow network using the node-splitting logic from Part A.

- Sources & Sinks: Add a Supersource S and a Supersink T .
- Connect s to the V_{in} node of all m starting points with capacity 1.
- Connect the V_{out} node of every boundary grid point to the supersink t with capacity 1.
- All grid connection edges (V_{out} , V_{in}) have capacity 1.

Algorithm: Run the Ford-Fulkerson method (or Edmonds-Karp) on this new graph. If the maximum flow equals m , then all m starting points successfully escaped.

Problem 24-1: Part B Solution (Time Complexity)

Time Complexity Analysis:

- The grid has $N = n^2$ vertices. After splitting, $|V| = 2n^2 + 2 = O(n^2)$
- Each grid node connects to at most 4 neighbors. Total edges $|E| = O(n^2)$
- The maximum possible flow $|f|$ is exactly m (the number of starting points). Since $m < n^2$, the max flow is relatively small.
- Using the standard Ford-Fulkerson time complexity of $O(E|f^*|)$:
- $O(E|f^*|) = O(n^2 m)$
- Since $m < n^2$, the worst-case running time is $O(n^4)$. This is highly efficient!